ChatGPT

Get Plus

Share

Given an array of integers and a target value, return the indices of the two numbers such that they add up to the target.

You may assume that each input has exactly one solution, and you may not use the same element twice.

Input:
n = 5
arr = [2, 7, 11, 15, 3]
target = 9

Output:
0 1

This is the classic **Two Sum** problem. The most efficient way to solve it is using a hash map (dictionary) to store values you've already seen.

Idea

As you iterate through the array:

- For each element `x`, compute `target - x` (the complement).
- Check if that complement already exists in the map.
- If yes → you've found the pair.
- If not → store the current number with its index.

C++ Code

cpp

Copy code

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    vector<int> arr = {2, 7, 11, 15, 3};
    int target = 9;

    unordered_map<int, int> mp; // value -> index

    for (int i = 0; i < n; i++) {
        int complement = target - arr[i];

        if (mp.find(complement) != mp.end()) {
            cout << mp[complement] << " " << i << endl;
            return 0;
        }

        mp[arr[i]] = i;
    }

    return 0;
}
```

Ask anything

ChatGPT

Share

👉 There will always be one valid pair

Get Plus ×

If you want, I can also turn this into a perfect "assignment-style" answer you can directly paste 👍

Give a step-by-step approach to solve this problem. Do not write code yet. Focus on logic and explanation. Also explain both brute force and optimized approaches.

Alright, let's focus purely on logic and build intuition step by step.

🧠 Step-by-Step Thinking

Goal:

Find two different indices such that:

Copy code

```
arr[i] + arr[j] = target
```

🐢 Brute Force Approach (Simple but Slow)

Idea:

Check **every possible pair** of numbers.

Steps:

1. Start from the first element (index `i = 0`)
2. Compare it with every other element (`j = i + 1` to end)
3. For each pair:
   - Add `arr[i] + arr[j]`
   - If it equals the target → return indices `i` and `j`
4. If not found, move `i` forward and repeat

Example Walkthrough

Array: `[2, 7, 11, 15, 3]`, Target = 9

- Compare:
  - (2, 7) → 9 ✅ → Answer found

Key Insight:

You are **trying all combinations** until you find the correct one.

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

Now write a clean and simple C++ program for this problem using both brute force and optimized approach. Use comments to explain each step.

Here's a clean and simple C++ program showing both approaches with clear comments.

Brute Force Approach

.cpp

Copy code

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    vector<int> arr = {1, 2, 3, 4, 5};
    int target = 7;

    // Try every possible pair
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {

            // Check if the sum equals target
            if (arr[i] + arr[j] == target) {
                cout << "Brute Force Output: " << i << " " << j << endl;
                return 0; // stop after finding the answer
            }
        }
    }

    return 0;
}
```

Optimized Approach (Using Hash Map)

.cpp

Copy code

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    vector<int> arr = {1, 2, 3, 4, 5};
    int target = 7;

    // Map to store number and its index
    unordered_map<int, int> mp;

    // Traverse the array
    for (int i = 0; i < n; i++) {

        int complement = target - arr[i]; // number we need

        // Check if complement already exists
        if (mp.find(complement) != mp.end()) {
            cout << "Optimized Output: " << mp[complement] << " " << i << endl;
            return 0;
        }

        // Store current element and its index
        mp[arr[i]] = i;
    }

    return 0;
}
```

Key Points (for your understanding or writing)

- Brute Force checks all pairs → simple but slow
- Optimized uses a map → fast and efficient
- Both return indices, not values

If you want, I can also:

- ↳ Combine both approaches in one program
- ↳ Add user input version
- ↳ Format it exactly for your assignment submission 🔥

- ↪ Add user input version
- ↪ Format it exactly for your assignment submission 👍

Compare the time complexity and space complexity of both approaches. Which one is better and why?

🧮 Time Complexity Comparison

🐢 **Brute Force**

- You use **two loops**:
  - Outer loop runs `n` times
  - Inner loop runs up to `n` times
- Total operations ≈ `n × n`

👉 **Time Complexity: $O(n^2)$**

⚡ **Optimized (Hash Map)**

- You traverse the array **only once**
- Each lookup in the map takes **O(1)** on average

👉 **Time Complexity: O(n)**

🧠 Space Complexity Comparison

🐢 **Brute Force**

- No extra data structures used

👉 **Space Complexity: O(1)** (constant space)

⚡ **Optimized**

- You store elements in a hash map

👉 **Space Complexity: O(n)**

🏆 Which One is Better?

✅ **Optimized Approach is Better**

Why?

- **It is much faster**, especially for large inputs
- Reduces time from **$O(n^2) \rightarrow O(n)$**
- Avoids unnecessary repeated comparisons

🤔 Trade-off

Factor	Brute Force	Optimized

Ask anything

ChatGPT

formatting 👍

Get Plus

Share

Test the optimized code with the sample input step by step and explain how the output is obtained.

Let's **dry run the optimized approach step by step** so you clearly see how the answer is formed.

🔢 Given Input

```ini
arr = [2, 7, 11, 15, 3]
target = 9
```

We use:

- **A map (mp)** → stores: `number → index`

🚶 Step-by-Step Execution

👉 Step 1 (i = 0)

- Current element = `2`
- Needed value (complement) = `9 - 2 = 7`

🔍 Check map:

- Is `7` in map? ❌ No

📦 Store current element:

```ini
mp = {2 : 0}
```

👉 Step 2 (i = 1)

- Current element = `7`
- Needed value = `9 - 7 = 2`

🔍 Check map:

- Is `2` in map? ✅ Yes!

🎯 Match found:

- `mp[2] = 0`
- Current index = `1`

✅ Final Output

Copy code

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

- Store values as we go
- Instantly check if the required number already exists

That's why it's fast (O(n)).

If you want, I can → convert this into a perfect handwritten-style explanation for your screenshots 👍

What are the edge cases for this problem? Update the optimized code to safely handle them.

Good question—this is where most real solutions break if you're not careful.

⚠️ Important Edge Cases

1. 1 Array size less than 2

- You need at least two elements
- Otherwise, no pair can exist

---

2. 🔁 Duplicate numbers

Example:

```
nums = [3, 3], target = 6
```

- Both numbers are same but indices are different
- Your solution must handle this correctly

---

3. 🚫 No valid pair exists

- Even though the problem says one solution exists, in real scenarios this might not be true
- Your code doesn't handle it safely (instead of crashing)

---

4. ➖ Negative numbers

Example:

```
nums = [-3, -1, -5, -2], target = -4
```

- Your logic should work for negatives as well

---

5. 🔢 Large numbers (overflow risk)

- If numbers are very large, `target - nums[i]` could overflow in extreme cases
- Using `long long` is safer

---

6. 🔄 Same element used twice (not allowed)

Example:

```
nums = [2], target = 4
```

- Cannot use the same index twice
- Our logic already avoids this, but it's worth noting

✅ Updated Optimized C++ Code (Safe Version)

```
#include <unordered_map>
using namespace std;

int* twoSum(int* nums, int numsSize, int target, int* returnSize) {
    int* res = NULL;
    int* cache = NULL;

    // Edge case: less than 2 elements
    if (numsSize < 2) {
        printf("Error: Need at least two elements\n");
        *returnSize = 0;
        return NULL;
    }

    unordered_map<int, int> map = {NULL};
    int* temp = NULL;
    int i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;
    int* ret = (int*)0;

    int* tmp = (int*);
    int* ptr = (int*);

    unordered_map<long long, long long> map; // number -> index

    for (i = 0; i < n; i++) {

        long long complement = target - nums[i];

        // Check if complement exists
        if (map == (long long*0, 1) && map[complement] != 0) {
            cache = map[complement] - 1;
            break;
        }

        // Store current element AFTER checking
        // (prevents using same element twice)
        map[nums[i]] = i;
    }

    // Edge case: no solution found
    printf("Error: No valid pair exists\n");
    *returnSize = 0;
    return NULL;
}
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

If you want, I can also give:

- ↳ Test cases to try
- ↳ Interview-style tricky variations
- OR ↳ a version that returns values instead of indices 🔥

Give the final version of the optimized C++ code with proper formatting, comments, and best practices.

Here's a **clean, final, and well-structured optimized C++ solution** with proper comments and best practices:

✅ Final Optimized C++ Code (Two Sum)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    // Edge case: need at least two elements
    if (n < 2) {
        cout << "No valid pair exists\n";
        return 0;
    }

    vector<long long> arr(n);

    // Input array elements
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    long long target;
    cin >> target;

    // Hash map to store value -> index
    unordered_map<long long, int> mp;

    // Traverse the array
    for (int i = 0; i < n; i++) {

        // Calculate required complement
        long long complement = target - arr[i];

        // Check if complement already exists
        if (mp.find(complement) != mp.end()) {
            cout << mp[complement] << " " << i << "\n";
            return 0;
        }

        // Store current element after checking
        // (prevents using same element twice)
        mp[arr[i]] = i;
    }

    // If no valid pair is found
    cout << "No valid pair exists\n";

    return 0;
}
```

🧠 Best Practices Used

- ✅ Fast I/O (`ios::sync_with_stdio(false); cin.tie(nullptr);`)
- ✅ Edge case handling (`n < 2`)
- ✅ Used `long long` for safety with large numbers
- ✅ Clear variable naming (`complement`, `mp`)
- ✅ Early return when solution is found
- ✅ Safe fallback output if no pair exists

⚡ Complexity

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.